

Pontificia Universidad Católica de Valparaíso
Facultad de Ingeniería
Escuela de Ingeniería Informática

**DISEÑO E IMPLEMENTACIÓN DE ALGORITMO
DE BÚSQUEDA CUCKOO PARA AJUSTE DE
FUNCIONES DE SELECCIÓN**

JAVIER EDUARDO FLORES VILCHES

Profesor Guía: Ricardo Soto De Giorgis

Profesor Co-referente: Broderick Crawford Labrin

Carrera: Ingeniería Civil Informática

Abril 2015

A quienes han estado junto a mi desde siempre, dándome su apoyo, cariño y amor incondicional, quienes siempre están y estarán presentes en mi corazón.

A quien ha dado sentido y razón a mi futuro, quien me ayuda a superar cada obstáculo e ilumina mis días con su amor.

A todos ellos gracias.

Resumen

La programación con restricciones es un paradigma enfocado a la resolución de problemas de satisfacción de restricciones. Este tipo de problema se modela utilizando una secuencia de variables asociada a un dominio y restricciones sobre estas variables. El método de resolución se basa en la exploración de un árbol de búsqueda utilizando estrategias de enumeración y propagación. Ambas estrategias son clave en el desempeño de la resolución, no obstante seleccionar la correcta es complejo debido a la dificultad de predecir su comportamiento. Búsqueda Autónoma propone una solución que permite realizar la elección de estrategias en forma automática en base a una serie de indicadores de desempeño y sin la necesidad de intervención humana. Sin embargo la obtención de buenos resultados utilizando búsqueda autónoma depende de la correcta configuración y optimización de sus componentes, para ello se propone en este trabajo la utilización de la metaheurística denominada “*Cuckoo Search*”.

Palabras claves: Programación con restricciones, Búsqueda Autónoma, Funciones de Selección, *Cuckoo Search*, Vuelos de Lévy, Meta heurísticas.

Abstract

Constraint programming is a paradigm focused on the resolution of constraint satisfaction problems. This kind of problem is modeled by using a sequence of variables associated to a domain and constraints over these variables. The resolution method is based on the exploration of a search tree by using enumeration and propagation strategies. Both strategies are key elements on the resolution performance, however selecting the correct one is complex as their behavior is hard to predict. Autonomous Search proposes to perform the strategy selection in an automatic way by employing a set of performance indicators without human interaction. Nevertheless, obtaining good results using autonomous search depends on the correct configuration and optimization of its components, to this task we propose in this work the use of the Cuckoo Search metaheuristic.

Keywords: Constraint Programming, Autonomous Search, Choice Function, *Cuckoo Search*, Lévy Flight, Meta-heuristics.

Índice

Resumen.....	iii
Abstract.....	iv
1 Introducción y Descripción del Problema	1
2 Definición del Proyecto.....	3
2.1 Objetivos.....	3
3 Estado del Arte.....	4
4 Programación con Restricciones	5
4.1 Resolución Mediante CSP	6
4.2 Generate and Test	6
4.3 Backtracking	7
4.4 Tecnicas de Consistencia	8
4.4.1 Node Consistency	8
4.4.2 Arc Consistency.....	8
4.4.3 Hyper Arc Consistency	8
4.5 Fordward Checking	8
4.6 Mantaining Arc Consistency.....	9
4.7 Estrategias de Enumeración	10
4.7.1 Heurísticas de Selección de Variable	11
4.7.2 Heurísticas de Selección de Valor	11
5 Búsqueda Autónoma	12
6 Cuckoo Search	14
6.1 Vuelos de Lévy.....	14
6.2 El Algoritmo Cuckoo Search	15
7 Implementación Algoritmo Cuckoo Search.....	16
8 Adaptación del Algoritmo Propuesto	18
8.1 Procesos de Entrada y Salida	18
8.2 Fases del Algoritmo.....	19
9 Pruebas	23

9.1	Pruebas de Ejecución	24
9.2	Pruebas Sobre las Estrategias de Enumeración	26
9.3	Pruebas Sobre Cuckoo Search y Comparaciones	37
10	Conclusiones	40
11	Referencias.....	41

Índice de Figuras

Figura 4.1 Resolución del problema 4-reinas usando algoritmo GT	6
Figura 4.2 Resolución del problema 4-reinas usando BT	7
Figura 4.3 Resolución del problema 4-reinas usando FC.....	9
Figura 4.4 Resolución del problema 4-reinas usando MAC	10
Figura 5.1 Arquitectura de cuatro componentes	12
Figura 7.1 Estructura Algoritmo Cuckoo Search.....	17
Figura 8.1 Población inicial.....	20
Figura 8.2 Fase de mejoramiento de soluciones	21
Figura 8.3 Búsqueda de nuevas soluciones	22
Figura 9.1 Función nextDouble().....	23

Índice de Tablas

Tabla 8.1 Parámetros “Cuckoo Search”	18
Tabla 9.1 Función nextGaussian.....	24
Tabla 9.2 Funciones de Selección.....	24
Tabla 9.3 Parámetros de la CF.....	25
Tabla 9.4 Comparación de CF	25
Tabla 9.5 Heurísticas de Selección de Variable.....	27
Tabla 9.6 Heurísticas de Selección de Valor.	27
Tabla 9.7 “Solving Time” Queen N.	28
Tabla 9.8 “BackTracks” (TB) Queen N.	29
Tabla 9.9 “Solving Time” Magic Square.	30
Tabla 9.10 “BackTracks” (TB) Magic Square.....	31
Tabla 9.11 “Solving Time” y “BackTracks” (TB) Latin Square.	32
Tabla 9.12 “Solving Time” y “BackTracks” (TB) Sudoku.	33
Tabla 9.13 “Solving Time” y “BackTracks” (TB) Quasi Group.	34
Tabla 9.14 “Solving Time” y “BackTracks” (TB) Knight Tour.....	35
Tabla 9.15 “Solving Time” y “BackTracks” (TB) Langford.....	36
Tabla 9.16 Resultados y comparación de “Solving Time” entre algoritmos.	38
Tabla 9.17 Resultados y comparación de “Back Tracks” entre algoritmos.....	39

1 Introducción y Descripción del Problema

La Programación con Restricciones o en inglés “*Constraint Programing*” (CP) es un paradigma de programación dedicado a resolver de forma eficiente problemas basados en restricciones. Este paradigma se centra en representar los problemas como una serie de variables asociadas a un dominio y las relaciones existentes entre ellas que son llamadas restricciones [1,2].

La forma de resolver los problemas se basa en la búsqueda de los valores asociados a cada variable, tales que se encuentren dentro del dominio definido y que cumplan con las restricciones previamente establecidas. Esta solución se logra generando un árbol de búsqueda que contiene todas las posibles soluciones, este árbol es recorrido por un motor de búsqueda denominado “*Solver*”.

La búsqueda es realizada alternando entre dos fases principales, la de Enumeración y la de Propagación. En la fase de Enumeración se selecciona un valor para una de las variables, y en base a esta se genera una rama del árbol de búsqueda. En la fase de Propagación se aplican propiedades de consistencia para podar las ramas que no llevan a soluciones potenciales, evitando realizar búsquedas innecesarias [1].

Ambas fases poseen diferentes formas de llevarse a cabo, contando con estrategias de enumeración y de propagación, que en conjunto forman la estrategia de resolución. La selección de una estrategia de resolución adecuada afecta directamente la capacidad de resolver un problema dado, no obstante seleccionar la estrategia adecuada es complejo dado que no es posible en general predecir su comportamiento .

La Búsqueda Autónoma o en inglés “*Autonomous Search*” (AS) es una técnica que permitiría a un “*Solver*” seleccionar automáticamente las estrategias de resolución más prometedoras para un problema. Esto es logrado mediante el uso de una función de selección llama en inglés “*Choice Function*” (CF) que en base a indicadores elige las mejores estrategias para cada situación [3].

Los indicadores son variables que representan el estado actual del proceso de búsqueda, por ejemplo pueden especificar la cantidad de pasos que se han realizado, o la profundidad del árbol de búsqueda. Cada problema a resolver es distinto y debido a esto los indicadores tienen mayor o menor relevancia a la hora de indicar el estado actual del proceso de búsqueda [1].

Definir la relevancia de cada indicador es un aspecto fundamental a la hora de optimizar el proceso de búsqueda, ya que dar mayor importancia a los indicadores correctos permite tener una CF mejor calibrada, esto lleva a una correcta elección de estrategias de resolución, lo que finalmente se traduce en una exploración más eficiente del árbol de búsqueda.

Este trabajo apunta a desarrollar un optimizador meta heurístico basado en el algoritmo de optimización “*Cuckoo Search*” (CS) para la Búsqueda Autónoma en la Programación con Restricciones.

“*Cuckoo Search*” es un algoritmo de optimización propuesto por Xin-She Yang y Suash Deb en 2009. Se encuentra inspirado en el comportamiento reproductivo de ciertas especies de cuckoos que depositan sus huevos en los nidos de otras especies. Los huevos representan potenciales soluciones, los mejores huevos pasan a la siguiente generación y los peores son descartados para luego ser reemplazados [4].

En la sección 2 de este informe se definirá el proyecto, sus alcances y los objetivos a cumplir, la sección 3 presenta el estado del arte, en la sección 4 se realiza una introducción a la programación con restricciones y su aplicación a la resolución de problemas, la sección 5 detalla como la búsqueda autónoma puede mejorar el proceso de búsqueda, la sección 6 introduce el algoritmo “*Cuckoo Search*” que será implementado como optimizador meta heurístico, la sección 7 detalla el proceso y la implementación del algoritmo en la arquitectura presentada, la sección 8 ilustra el proceso de adaptación del algoritmo, la sección 9 presenta los resultados obtenidos luego de realizar múltiples pruebas sobre problemas clásicos y finalmente la sección 10 presenta las conclusiones obtenidas.

2 Definición del Proyecto

A continuación se detallan los objetivos, y la planificación del proyecto de implementación del algoritmo de optimización “*Cuckoo Search*” para Búsqueda Autónoma en Programación con Restricciones.

2.1 Objetivos

Objetivo General:

- Implementar un optimizador para búsqueda autónoma mediante “*Cuckoo Search*”.

Objetivos Específicos:

- Comprender el funcionamiento de la arquitectura para búsqueda autónoma.
- Implementar un optimizador para funciones de selección en Búsqueda Autónoma mediante “*Cuckoo Search*”.
- Realizar experimentos con el nuevo optimizador sobre un conjunto de problemas clásicos.
- Comparar los resultados obtenidos con otros optimizadores.

3 Estado del Arte

La resolución de problemas combinatorios complejos ha sido una importante contribución por parte de la computación. La Programación con Restricciones es un paradigma que apunta a resolver estos problemas de forma eficiente, utilizando principios como la investigación de operaciones, el análisis numérico, inteligencia artificial y los lenguajes de programación. La Programación con Restricciones tiene múltiples aplicaciones como expresar coherencia geométrica en los gráficos por computador, diseño de ingeniería para estructuras mecánicas complejas, asegurar y/o recuperar consistencia en sistemas de bases de datos, localización de fallas en ingeniería eléctrica e incluso secuenciando ADN en biología molecular [2].

El estudio de las Estrategias de Enumeración es fundamental para Programación con Restricciones ya que la correcta elección de estrategias afecta drásticamente el proceso de resolución del problema. Debido a la compleja tarea de seleccionar las estrategias correctas para cada problema, existe una tendencia a automatizar el proceso de selección en base a un análisis de datos obtenidos durante el proceso de resolución.

Esta tendencia ha llevado a la aparición de propuestas como el Motor de Restricciones Adaptativa "*Adaptive Constraint Engine*" (ACE) que corresponde a un framework que aprende heurísticas de ordenamiento en base a la experiencia recopilada durante el proceso de resolución [5].

Autonomous Search es una técnica que propone una solución al problema de seleccionar Estrategias de Enumeración al reemplazar las que presentan un peor desempeño por aquellas más prometedoras durante el proceso de resolución se busca mejorar el desempeño [6]. La integración entre CP y AS es una tendencia que ha sido propuesta en forma de frameworks teóricos [7] y arquitecturas [8,9].

"*Cuckoo Search*" es un algoritmo de optimización propuesto por Xin-She Yang y Suash Deb en 2009. Se basa en el comportamiento reproductivo de algunas especies de cuckoos que depositan sus huevos en nidos de otras especies. Este algoritmo ha demostrado su potencial y ha sido utilizado para resolver múltiples problemas como por ejemplo el diseño de resortes, asignación de turnos de enfermería entre otros [4,10].

4 Programación con Restricciones

La Programación con Restricciones (CP) es un poderoso paradigma de la programación, dedicado a resolver de forma eficiente problemas basados en restricciones [6].

En CP los problemas son modelados en base a variables y restricciones, las variables poseen un dominio y representan las incógnitas del problema, mientras que las relaciones entre ellas son representadas como restricciones. La solución a un problema en CP es encontrada una vez que se obtiene valores para cada variable, pertenecientes al dominio y que cumplan con todas las restricciones planteadas.

La forma en que se resuelven los problemas es construyendo un árbol de búsqueda que contenga las potenciales soluciones, esto es logrado intercalando los dos procesos principales llamados: fase de Enumeración y fase de Propagación.

En la fase de Enumeración se escoge una variable y un valor perteneciente a su dominio, luego a partir de esta se crea una rama del árbol de búsqueda. La elección de la variable y su valor son determinadas por la Heurística de Ordenamiento de Variables y la Heurística de Ordenamiento de Valores. Y en conjunto son llamadas como Estrategias de Enumeración.

La fase de Propagación aplica propiedades de consistencia para podar el árbol, esto implica identificar valores que no llevan a soluciones viables, eliminándolos temporalmente a fin de evitar que el proceso de búsqueda inspeccione soluciones imposibles.

El conjunto formado por las Estrategias de Enumeración y las Técnicas de Propagación es conocido como la Estrategia de Resolución. La elección de una Estrategia de Resolución es crítica ya que tiene un efecto directo en el proceso de resolución del problema, es por esto que la elección debe ser basada en las características del problema y las posibles estrategias aplicadas a cada caso, esta compleja decisión requiere una amplia experiencia.

La representación de un problema mediante las técnicas expuestas es llamada Problema de Satisfacción de Restricciones “*Constraint Satisfaction Problem*” (CSP) a continuación se detalla el proceso de resolución de problemas usando este paradigma.

4.1 Resolución Mediante CSP

Un Problema de Satisfacción de Restricciones (CSP) puede ser definido como $P = (X, D, C)$ donde:

- X es una n -tupla de las variables $X = (x_1, x_2, \dots, x_n)$,
- D representa la n -tupla de dominios $D = (D_1, D_2, \dots, D_n)$ tal que $x_i \in D_i$ y que D_i es un conjunto de valores para $i = 1, \dots, n$.
- C corresponde a la t -tupla de restricciones tal que $C = (C_1, C_2, \dots, C_t)$.

El enfoque de CSP es buscar la primera solución que satisfaga todas las restricciones planteadas en el problema, y para esto es necesario encontrar valores apropiados para cada una de las variables [1].

4.2 Generate and Test

Uno de los métodos más sencillos para resolver un problema de CSP es usar un algoritmo de “*Generate and Test*” (GT), este método consiste probar sistemáticamente las posibles soluciones hasta encontrar una que cumpla con todas las restricciones. Este método genera las posibles soluciones realizando todas las combinaciones existentes entre los dominios de cada variable [2].

A modo de ejemplo podemos considerar el problema n -reinas (n -queens), dado un tablero de ajedrez de tamaño $n \times n$ se busca posicionar n reinas de forma que no compartan filas, columnas o diagonales.

La Figura 4.1 representa la búsqueda de solución al problema 4-reinas aplicando el algoritmo GT.

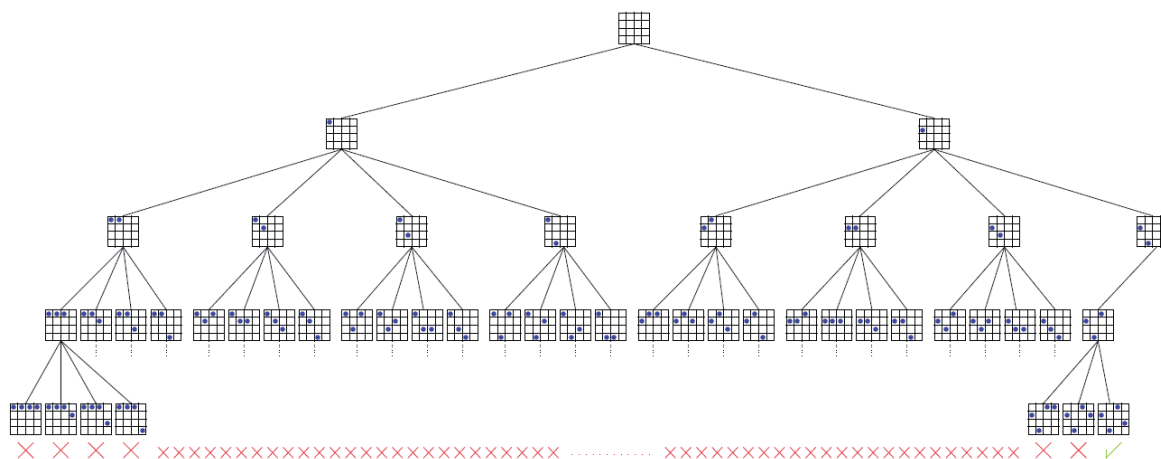


Figura 4.1 Resolución del problema 4-reinas usando algoritmo GT

El proceso empleado por el algoritmo GT consta en verificar si las restricciones se cumplen una vez que todas las variables han sido instanciadas, por lo tanto si una solución parcial no cumple una de las restricciones, no será detectada hasta que todas las variables sean instanciadas. Este proceso lleva la generación y prueba de muchas soluciones erróneas que podrían haber sido detectadas y eliminadas previamente [2].

Este algoritmo es simple de implementar pero el costo de realizar la búsqueda es proporcional al número total de posibles soluciones, por lo que no ofrece buenos resultados al enfrentarse a problemas de mayor tamaño.

4.3 Backtracking

Otra forma de resolver problemas de CSP es mediante “*Backtracking*” (BT), este método al igual que GT realiza una búsqueda sistemática hasta encontrar una solución adecuada. Las potenciales soluciones son generadas seleccionando un valor para una variable y comprobando las restricciones en cuanto todas las variables involucradas en una restricción son instanciadas. Esto permite detectar cuando una solución parcial viola una restricción, por lo tanto es descartada y el algoritmo regresa a la última variable instanciada que aun tenga valores alternativos para probar [2].

La Figura 4.2 ilustra el proceso de búsqueda realizado por BT para resolver el problema 4-reinas. A diferencia de GT es posible detectar violaciones a las restricciones en cuanto dos variables son instanciadas, reduciendo así el espacio de búsqueda.

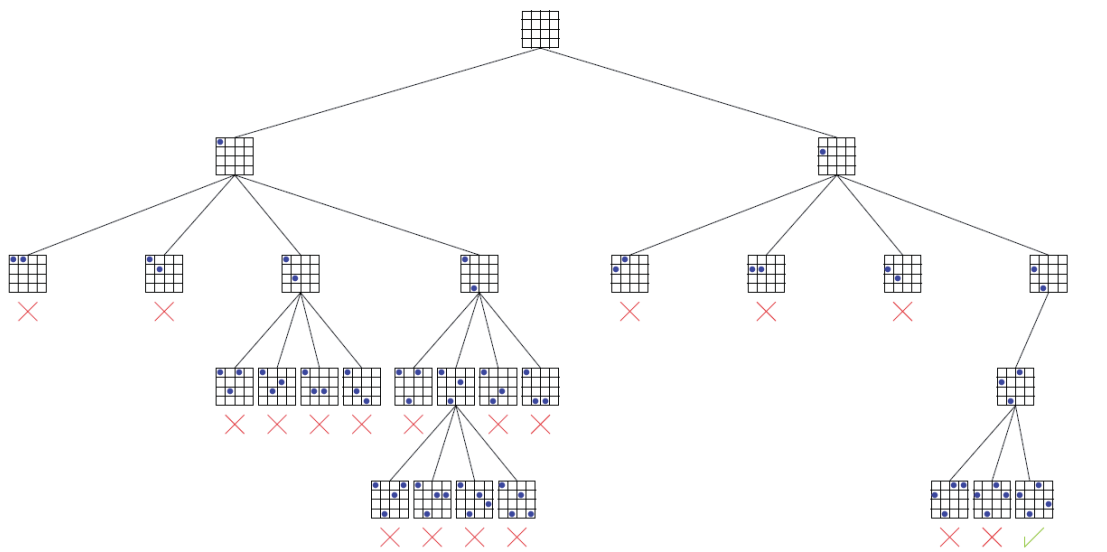


Figura 4.2 Resolución del problema 4-reinas usando BT

4.4 Tecnicas de Consistencia

A fin de mejorar el desempeño del proceso de búsqueda se aplican técnicas de consistencia que permitan reducir el espacio de búsqueda. Esto se logra realizando un proceso llamado propagación de restricciones mediante el cual se aplica un determinado nivel de consistencia entre los sub conjuntos de variables.

4.4.1 Node Consistency

“*Node Consistency*” es la técnica utilizada para aplicar consistencia cuando se trabaja con restricciones unarias (restricciones que involucran solo una variable). La condición de consistencia se cumple cuando para cada valor en el actual dominio de una variable x , las restricciones unarias sobre x son respetadas. En caso que el dominio de x contenga valores que no respetan las restricciones, entonces se considera que no cumple con la consistencia y puede ser eliminado removiendo el valor de x que no cumple la restricción [2].

4.4.2 Arc Consistency

En el caso de las restricciones binarias (restricciones que involucran dos variables) se aplica “*Arc Consistency*”, y la condición de consistencia se cumple si para una restricción que afecta las variables x e y cada valor perteneciente al dominio de ambas es parte de una posible solución. Por lo tanto un CSP es “*Arc Consistency*” si todas las restricciones binarias son consistentes [2].

4.4.3 Hyper Arc Consistency

El concepto detrás de “*Arc Consistency*” puede ser generalizado como “*Hyper Arc Consistency*” y esta condición se cumple si cada valor perteneciente a cada dominio involucrado en una restricción es parte de una potencial solución.

4.5 Forward Checking

Si bien BT aparenta ser una mejora en relación con GT, ambos métodos requieren que una variable sea instanciada antes de poder revisar si cumplen o no con las restricciones. “*Forward Checking*” (FC) busca evaluar las soluciones aun antes de haber instanciado ciertas variables, para lograr esto FC aplica el concepto de “*Arc Consistency*” esto permite quitar momentáneamente los valores que causarán conflicto. Esto permite al algoritmo saber si una solución parcial es o no consistente, reduciendo el espacio de búsqueda [2].

La Figura 4.3 muestra el proceso mediante el cual valores pertenecientes a los dominios son removidos. Por ejemplo la reina es posicionada en (1,1) e inmediatamente se quitan todos los valores de la misma fila y diagonal. En el siguiente nodo se posiciona la reina en el espacio disponible (3,2) lo que resulta inconsistente ya que no quedan valores disponibles para la tercera reina. A partir de este punto el algoritmo sigue la misma lógica hasta encontrar la solución.

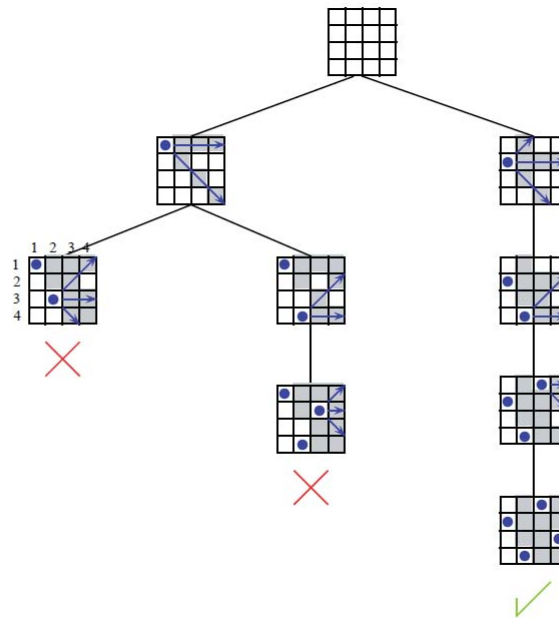


Figura 4.3 Resolución del problema 4-reinas usando FC

4.6 Maintaining Arc Consistency

“*Forward Checking*” busca los conflictos entre los valores actuales y los futuros, pero esta técnica puede ampliarse aún más si se comprueba la presencia de conflictos entre los futuros valores. Esta técnica es llamada “*Maintaining Arc Consistency*” (MAC).

La Figura 4.4 muestra el proceso mediante el cual el algoritmo MAC reduce el espacio de búsqueda aun antes de lo logrado por FC, pero implicando una mayor carga de trabajo a cada asignación de una variable.

Se comienza con la reina en la posición (1,1) las flechas azules representan los conflictos entre los valores actuales y los futuros que son removidos. Luego se verifican los posibles conflictos entre las futuras soluciones comenzando por la posición (3,2). Se demuestra que la posición (3,2) no es consistente ya que no deja espacio para la tercera reina (flechas de color naranja). El algoritmo continua con la siguiente alternativa en la celda (4,2) la cual solo deja como alternativa para la tercera reina la posición (2,3), pero esta es marcada como inconsistente ya que no deja lugar para una cuarta reina. El algoritmo continuará hasta encontrar una solución válida.

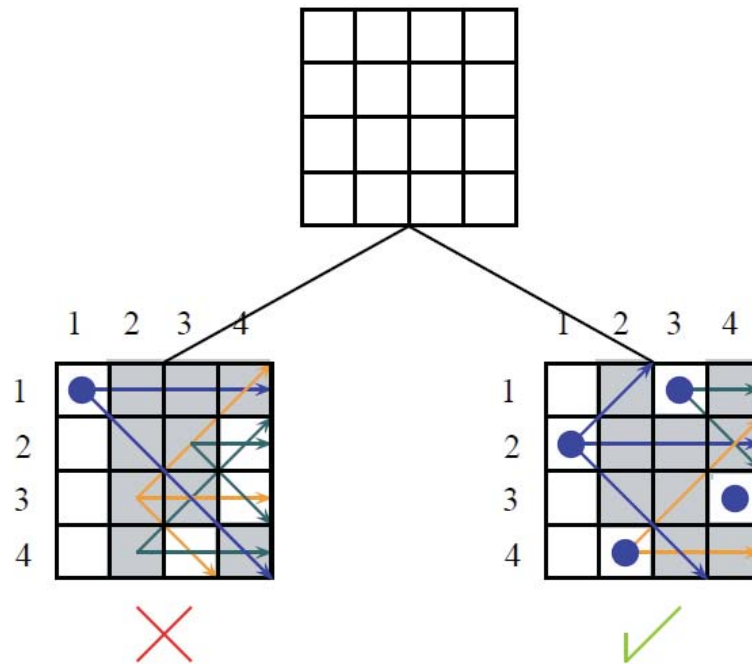


Figura 4.4 Resolución del problema 4-reinas usando MAC

4.7 Estrategias de Enumeración

Hasta el momento solo se ha considerado que al momento de seleccionar una variable (por ejemplo la primera reina en un problema de n -reinas), se le asigna el primer valor de su dominio para comenzar a resolver. El orden en que estos valores son seleccionados (La variable y el valor de su dominio) afectan el desempeño del proceso de resolución del problema [2].

El proceso de selección de una variable y su valor se les llama Heurísticas de Selección de Variable y Heurísticas de Selección de Valor y en conjunto forman las Estrategias de Enumeración (EE). Estas estrategias pueden ser tanto estáticas como dinámicas, en el primer caso el orden es especificado antes de comenzar el proceso de búsqueda y en el caso dinámico la selección de la siguiente variable se realiza en base al estado actual del proceso de búsqueda.

Existen múltiples alternativas tanto para las Heurísticas de Selección de Variable como para las Heurísticas de Selección de Valor, por lo que definir una combinación adecuada de ellas es parte esencial de esta investigación, en la sección 9.2 se muestran los resultados de las pruebas realizadas sobre el conjunto de Heurísticas, estas pruebas demuestran cómo para cada problema planteado existe un conjunto de Heurísticas que obtienen el mejor desempeño.

4.7.1 Heurísticas de Selección de Variable

Las Heurísticas de Selección de Variable fijan en orden en que las variables son seleccionadas para luego asignar un valor de su dominio, existen diferentes alternativas para fijar este orden y entre las más comunes se encuentran las siguientes:

- “*First*”: Escoge la primera variable en la lista.
- “*Anti Minimum Remaining Values*” (AMRV): Escoge la variable con el dominio más grande.
- “*Minimum Remaining Values*” (MRV): Escoge la variable con el dominio más pequeño.
- “*Ocurrence*” (OC): Escoge la variable con mayor cantidad de restricciones.
- “*Smallest*” (Sm): Escoge la variable con el valor más pequeño de dominio,
- “*Largest*” (Lg): Escoge la variable con el mayor valor en el dominio.
- “*Most Constrained*” (MC): Escoge la variable con el dominio más pequeño y en caso de existir más de una, selecciona la que posee más restricciones.
- “*Max Regret*” (MR): Escoge la variable con la mayor diferencia entre el valor más pequeño y el más pequeño del segundo dominio.

Cada Heurística tiene un enfoque diferente para seleccionar variables, por ejemplo la Heurística “*Ocurrence*” selecciona la variable con mayor cantidad de restricciones debido a que al ser instanciadas se involucran un gran número de restricciones que deben ser respetadas, esto llevaría a una reducción del espacio de búsqueda.

4.7.2 Heurísticas de Selección de Valor

Las Heurísticas de Selección de Valor son aquellas encargadas de definir en orden en que se seleccionarán los valores pertenecientes al dominio de una cierta variable. A continuación se presentan tres ejemplos que son los utilizados durante esta investigación.

- “*Indomain Min*”: Escoge el elemento más pequeño del dominio.
- “*Indomain Middle*”: Escoge el elemento medio del dominio.
- “*Indomain Max*”: Escoge el elemento más grande del dominio.

5 Búsqueda Autónoma

La Búsqueda Autónoma es una nueva técnica que permite a un “*Solver*” realizar su propia elección de Estrategias de Resolución, dándole la posibilidad de mejorar su eficiencia sin la necesidad de un usuario experto ajustando los parámetros.

La Búsqueda Autónoma intenta dar al “*Solver*” la capacidad de administrar y adaptar el proceso de búsqueda de forma autónoma, utilizando indicadores y parámetros que le permitan identificar el desempeño del proceso de búsqueda y en base a esto realizar los ajustes necesarios para obtener mejores resultados [11].

Como ya fue mencionado anteriormente las Estrategias de Enumeración están compuestas por las Heurísticas de Ordenamiento de variables y valores, se puede decir entonces que el proceso de Enumeración es Adaptativo cuando el “*Solver*” es capaz de controlar y seleccionar automáticamente sus propias Heurísticas de Ordenamiento [6].

La capacidad de realizar los ajustes al mismo tiempo que se está resolviendo al problema, es lo que permite a la Búsqueda Autónoma mejorar el desempeño del “*Solver*”.

La utilización de una arquitectura de cuatro componentes representado en la Figura 5.1. [3] permite una selección dinámica de Estrategias de Enumeración basada en su desempeño durante la resolución. La arquitectura consta de cuatro componentes: “*Solve*”, “*Observation*”, “*Analysis*” y “*Update*” [6].

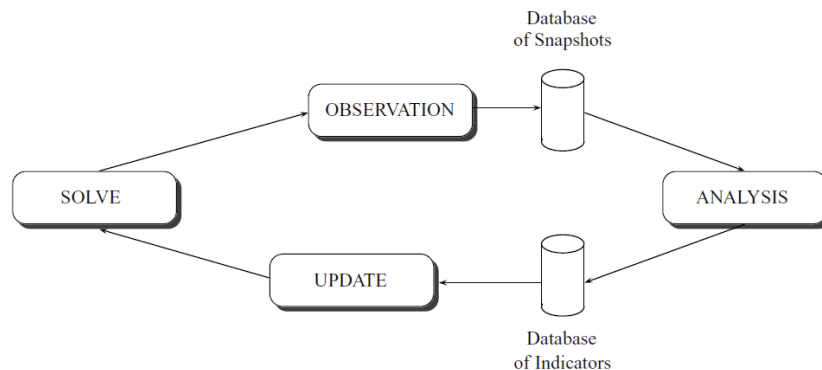


Figura 5.1 Arquitectura de cuatro componentes

El componente “*Solve*” utiliza un algoritmo de CP para realizar las fases de Enumeración y Propagación. A su vez las estrategias usadas son seleccionadas de un ranking basado en su desempeño que es administrado por el componente “*Update*”. El componente “*Observation*” se encarga de registrar información relevante sobre el proceso de resolución. El componente “*Analysis*” estudia la información recopilada por el componente “*Observation*” y le entrega indicadores al componente “*Update*” [8].

El componente “*Update*” está encargado de generar un ranking de las estrategias de acuerdo a su desempeño, esto se logra mediante una Función de Selección que haciendo uso de los indicadores evalúa el desempeño de cada estrategia. La Función de Selección f en un tiempo n para una Estrategia S_j se define como:

$$f_n(S_j) = \sum_{i=1}^l \alpha_i f_{i(n)}(S_j)$$

Donde l es el número de indicadores considerados y α es un parámetro que define la relevancia de cada indicador. El parámetro α es usualmente definido por un optimizador que define los valores más adecuados para una cierta Función de Selección [12].

Adicionalmente al valor α para controlar la relevancia de un indicador i para una cierta estrategia S_j en un periodo de tiempo, se utiliza una estrategia estadística llamada suavizado exponencial. Esto permite dar mayor relevancia y peso a las observaciones más recientes y quitárselo a las observaciones históricas, el efecto que esto tiene sobre el comportamiento es una reacción más rápida ante las nuevas observaciones y un menor peso a los datos previos.

$$f_{i(n)}(S_j) = \beta_i x_{n-1} + (1 - \beta_i) f_{i(n-1)}(S_j)$$

La ecuación representa en proceso de suavizado exponencial, donde el valor β es el encargado de dar más o menos importancia a las observaciones más recientes. El valor β se encuentra entre 0 y 1, por lo que dado un valor β cercano a 0 se le está dando menos importancia al valor histórico x_{n-1} y mayor a la observación más reciente, acelerando la capacidad de reacción [1].

6 Cuckoo Search

Actualmente muchos algoritmos meta heurísticos son inspirados por la propia naturaleza, por ejemplo “*Particle Swarm Optimization*” (PSO) fue inspirado por el comportamiento de enjambre de aves y peces. Lo que hace famosos a estos algoritmos, es que imitan las mejores características de la naturaleza, observando el comportamiento de sistemas biológicos que son el resultado de millones de años de evolución y selección natural [4].

Principalmente podemos encontrar dos características presentes en las metas heurísticas modernas: intensificación y diversificación. La intensificación hace referencia a la búsqueda y selección de mejor candidato, mientras que la diversificación asegura la capacidad de explorar todo el espacio de búsqueda de forma eficiente [4].

“*Cuckoo Search*” es un algoritmo basado en el comportamiento reproductivo de ciertas especies, y en conjunto con los Vuelos de Lévy sirven de inspiración para una prometedora meta heurística.

6.1 Vuelos de Lévy

Los Vuelos de Lévy representan la forma en que varios animales e insectos exploran el terreno utilizando una serie de vuelos en líneas rectas, marcados por unos abruptos cambios de dirección.

Los Vuelos de Lévy presentan básicamente un recorrido aleatorio, cuya longitud de desplazamiento se extrae de una distribución de Lévy que tiene una varianza y una media infinitas [13].

El éxito de los Vuelos de Lévy se basa en que un porcentaje de los nuevos movimientos se basan en la mejor solución obtenida hasta el momento, acelerando el proceso de búsqueda, pero asegurando que el porcentaje restante sea generado completamente al azar y lejos de solución actual a fin de asegurar una exploración eficiente y no queda atrapado en mínimos locales [14].

6.2 El Algoritmo Cuckoo Search

El Algoritmo “*Cuckoo Search*” fue presentado por Xin-She Yang, profesor de la universidad de Cambridge en 2009. Es un algoritmo basado en el comportamiento parasitario de los Cukoos a la hora de depositar sus huevos.

La madre cuckoo deposita sus huevos en un nido ajeno a la espera de que este no sea descubierto y que eventualmente sea criado por la madre sustituta, sin que esta sospeche la presencia de un intruso [10].

El algoritmo puede ser descrito de la siguiente forma:

- Cada cuckoo deposita un huevo en un nido seleccionado al azar.
- Los mejores nidos con los mejores huevos sobrevivirán y formaran la siguiente generación.
- Existe una cantidad fija de nidos disponibles y cada huevo tiene una probabilidad P_a perteneciente al dominio entre 0 y 1. De ser descubierto, en cuyo caso la madre abandona el nido y crea uno nuevo.

Siguiendo las reglas definidas anteriormente, el algoritmo de optimización puede ser resumido en el siguiente pseudocódigo [4].

Seudocódigo Cuckoo Search con Vuelos de Lévy

```
1:  Inicio
2:  Función objetivo  $f(x)$ ,  $x=(x_1,...x_d)^T$ 
3:  Generar población inicial de n nidos  $x_i$  ( $i=1,2,...,n$ )
4:  While ( $t < \text{MáximaGeneración}$ ) o (Criterio de finalizar)
5:    Obtener nuevo cuckoo mediante vuelo de lévy
6:    Evaluar su calidad / fitness  $F_i$ 
7:    Elegir nuevo nido entre n (ejemplo j) aleatoriamente
8:    Si ( $F_i > F_j$ ) /* '>' Se utiliza para maximizar, '<' para minimizar */
9:      Reemplazar j por la nueva solución
10: Fin
11:   Una fracción  $p_a$  de los peores nidos son abandonados
12:   Y nuevos nidos son construidos
13:   Mantener mejores soluciones
14:   Ordenar las soluciones y encontrar la mejor
15: Fin mientras
16: Fin
```

Se puede apreciar la sencillez y robustez de este algoritmo, debido a mínima cantidad de variables necesarias, y su capacidad para resolver problemas maximización y minimización, esto facilita su implementación aplicado a distintos problemas de forma sencilla.

7 Implementación Algoritmo Cuckoo Search

Previamente en la sección cuatro fue descrita la arquitectura de cuatro componentes para “*Autonomous Search*” propuesta, y dentro de esta la importancia del componente “*Update*” y su función de selección para realizar un listado de las estrategias de acuerdo a su desempeño.

La función de selección permite usar distintos indicadores para medir el desempeño de las estrategias, pero como ha sido mencionado antes no todos los indicadores representan igual relevancia para diferentes tipos de problema, debido a esto se maneja un valor α comprendido entre -100 y +100 que da la posibilidad de manejar la relevancia de cada indicador por separado.

Definir los valores α más adecuados para un cierto problema, puede ser presentado como un problema de optimización, ya que se busca encontrar un conjunto de valores α que mejoren los resultados obtenidos por la función de selección.

Queda clara la necesidad de utilizar una métrica que permita calcular la calidad de un conjunto de valores α , para esto se utiliza un indicador llamado Reducción del Espacio de Búsqueda (IN-3). En base a este indicador podemos definir qué conjunto de α es mejor, ya que a mayor IN-3 estamos frente a un mejor desempeño en la búsqueda.

El algoritmo “*Cuckoo Search*” representa las soluciones como Nidos o Cuckoos, y de acuerdo a la calidad de las soluciones, estas pueden ser mejoradas o eliminadas y reemplazadas por nuevas soluciones en cada ciclo o generación. Esto permite al algoritmo realizar búsquedas locales para mejorar soluciones prometedoras y reemplazar las soluciones menos prometedoras para mejorar el espacio de búsqueda.

El funcionamiento del algoritmo requiere de dos procesos importantes, el primero es enviar las soluciones propuestas, a fin de ser utilizadas y evaluadas por el “*Solver*”, para luego recibir el valor correspondiente al desempeño que presentaron cada una de las soluciones a fin de poder mejorarlas. El desempeño de un conjunto de α es representado por el indicador IN-3 y este indicador de desempeño representará el “*Fitness*” de cada solución.

Finalmente es necesario considerar un último indicador para implementar el algoritmo, ya que se busca encontrar el mejor conjunto de α , no tiene sentido ejecutar este algoritmo durante todo el proceso de búsqueda por varios motivos, dentro de ellos podemos destacar que las posibles mejoras a los valores de α que se realizan al estar cerca de encontrar una solución, tendrán poco o nulo impacto en el desempeño y peor aún podrían impactar negativamente el proceso debido al costo de ejecutar el algoritmo.

Para determinar el tiempo máximo de ejecución del algoritmo se utiliza un nuevo indicador llamado “*Cutoff*”, este permite definir el punto el que el algoritmo dejara de buscar mejores conjuntos de valores α , en ese momento se seleccionara la mejor solución encontrada y se continuará la búsqueda utilizando ese conjunto de α hasta el final.

P_a [	α_{11}	α_{12}	$\dots$	α_{1m}	f_1
	α_{21}	α_{22}	$\dots$	α_{2m}	f_2
	$\dots$		$\dots$		$\dots$
	α_{n1}	α_{n2}		α_{nm}	f_n

Figura 7.1 Estructura Algoritmo Cuckoo Search

La estructura base del algoritmo puede ser representada en la Figura 7.1 donde la matriz de n filas y m columnas representa las soluciones. Cada fila representa un conjunto de valores α por lo que el valor n es a su vez el tamaño de la población de cuckoos o Nidos en existencia. El valor m representa la dimensión de cada solución y por lo tanto es el valor α específico asignado a un indicador.

Cada una de las filas, o potenciales soluciones serán evaluadas para definir su desempeño, este será representado por el valor “*Fitness*” (f) permitiendo definir un ranking de acuerdo a la calidad de cada solución propuesta. Finalmente P_a el porcentaje de soluciones de menor calidad que serán eliminadas y reemplazadas por nuevas soluciones aleatorias, mientras que el resto de las soluciones intentaran ser mejoradas [15].

8 Adaptación del Algoritmo Propuesto

A continuación se especifica la implementación del algoritmo, definiendo claramente como fue adaptado en base a la arquitectura previamente presentada.

8.1 Procesos de Entrada y Salida

Luego de definir la estructura y la funcionalidad del algoritmo, este fue implementado en lenguaje de programación JAVA y adaptado para funcionar en conjunto con la arquitectura de cuatro secciones definida previamente en la sección 5. El proceso de adaptación comienza con la definición del proceso de comunicación entre la arquitectura y el algoritmo, dado este caso podemos identificar tres procesos que contemplan la retroalimentación existente entre ambos.

El primer proceso relevante de comunicación, corresponde a envío de los parámetros que el algoritmo utilizara para resolver el problema de optimización solicitado. Estos parámetros corresponden a los tanto valores configurables propios del algoritmo, como a los definidos por el problema en cuestión. La Tabla 8.1 describe los parámetros enviados al algoritmo.

Tabla 8.1 Parámetros “Cuckoo Search”

n	Población de Cuckoos/Nidos	Cantidad de soluciones con las que el algoritmo trabajará, cada una es un conjunto de valores α
d	Dimensión de un Cuckoo/Nido	Numero de indicadores para los cuales se necesita definir un valor α
P_a	Factor de descubrimiento	Porcentaje de los nidos que serán reemplazados
T	Máximo de iteraciones	Número máximo de iteraciones
lb	Límite Inferior	Límite inferior de los valores α
ub	Limite Superior	Límite superior de los valores α

Como fue explicado en las secciones 6.1 y 7 cada potencial solución requiere ser evaluada a fin de discriminar su calidad, este proceso lo lleva a cabo la arquitectura por lo que el segundo proceso de comunicación se realiza cuando el algoritmo envía las potenciales soluciones, en espera que éstas sean evaluadas y su calidad retornada en la forma de un valor llamado “*Fitness*”, esto permite al algoritmo continuar su proceso de búsqueda para encontrar soluciones de mejor calidad.

Finalmente, una vez que el algoritmo agota su tiempo de búsqueda, la mejor solución encontrada hasta el momento es retornada a la arquitectura y es usada para continuar la resolución del problema hasta el final de su ejecución.

8.2 Fases del Algoritmo

Tal y como ha sido definido, el algoritmo busca imitar el método de selección de nidos que utilizan ciertas aves y en base a esto obtener un proceso de búsqueda y selección que permita encontrar soluciones óptimas. El método utilizado puede ser resumido en tres fases que hacen uso de la búsqueda y el perfeccionamiento de las soluciones.

La primera fase corresponde a la generación inicial de soluciones, este proceso aleatorio genera un número n de soluciones que serán utilizadas como punto de partida para realizar la búsqueda. Una vez que las soluciones totalmente aleatorias han sido generadas, se procede a evaluar su calidad, a fin de encontrar el mejor candidato.

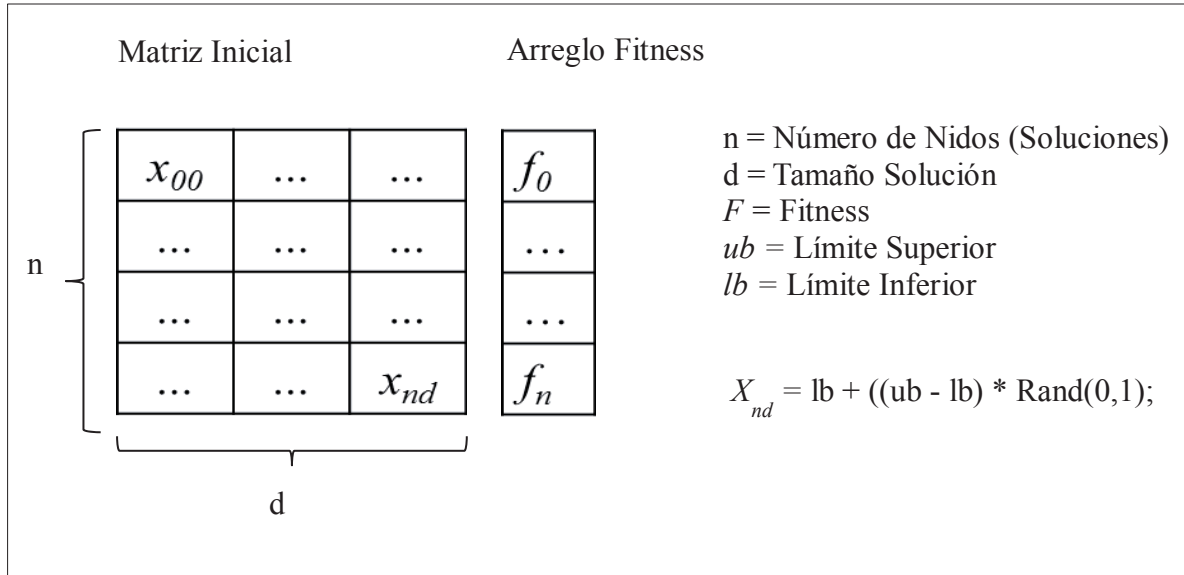


Figura 8.1 Población inicial

La Figura 8.1 muestra el funcionamiento de la primera etapa del algoritmo “*Cuckoo Search*” correspondiente a la generación de la población inicial, la variable n define la cantidad de soluciones o Cuckoos, mientras que d es la dimensión que cada una de estas soluciones posee. La generación de los valores aleatorios se realiza mediante una función “*Random*”, la que utiliza una semilla tomada del tiempo transcurrido para generar diferentes soluciones en cada ejecución, se utiliza los valores de los límites para generar automáticamente soluciones que cumplan con esta condición.

La siguiente fase corresponde al proceso de mejoramiento de las soluciones encontradas, esta fase puede ser considerada como un proceso de explotación, ya que se buscan nuevas soluciones en un área local cercana a una solución ya existente, tomando esta como punto de partida.

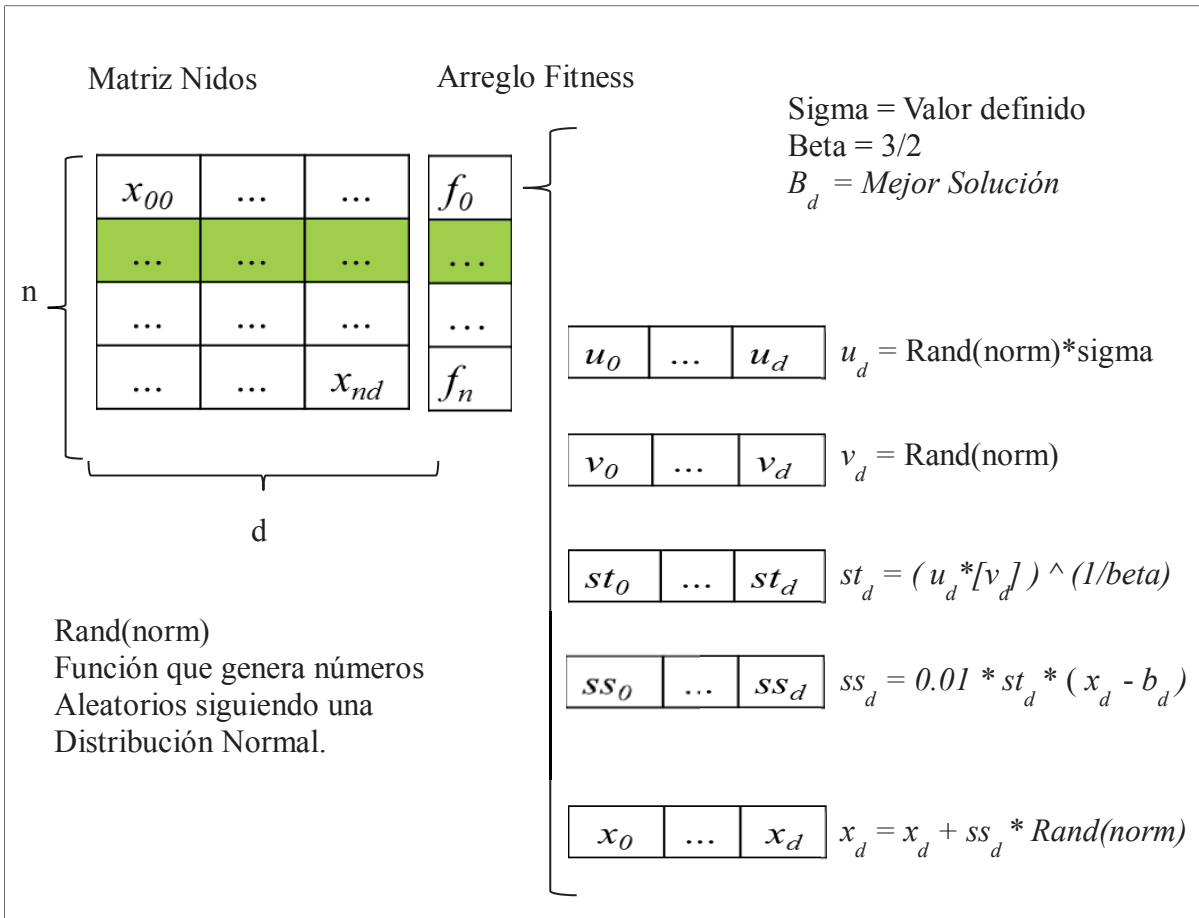


Figura 8.2 Fase de mejoramiento de soluciones

La Figura 8.2 ilustra el proceso de mejoramiento de soluciones mediante vuelos de Levy, este proceso se realiza para todas las soluciones salvo la que tiene el mejor “Fitness”.

El proceso de búsqueda local mediante vuelos de levy se realiza para cada una de las soluciones previamente encontradas, para esto se utiliza su actual posición. La generación de un “Stepsize” se hace mediante una implementación del algoritmo de Mantegna, luego se realiza un movimiento en cada dimensión generando una nueva solución cercana a la original. Antes de concluir se aplican una verificación de límites para evitar que se escape del espacio definido, para luego realizar una comparación entre las soluciones originales, y las nuevas obtenidas mediante este proceso, reemplazándolas por su mejora en caso de ser encontrada.

Finalmente, el último proceso corresponde a una búsqueda que permite explorar nuevas soluciones lejos de las originales, esto previene que el algoritmo se quede atrapado en un espacio local, complementando los procesos anteriores.

El algoritmo genera nuevas soluciones utilizando permutaciones aleatorias de las soluciones ya encontradas y utilizando valores aleatorios. A diferencia del proceso anterior, este lleva relación con el porcentaje de descubrimiento de nidos, ya que se utiliza el valor p_a para seleccionar sobre qué soluciones se realizara el proceso de búsqueda, ya que esto representa los nidos que son descubiertos y abandonados.

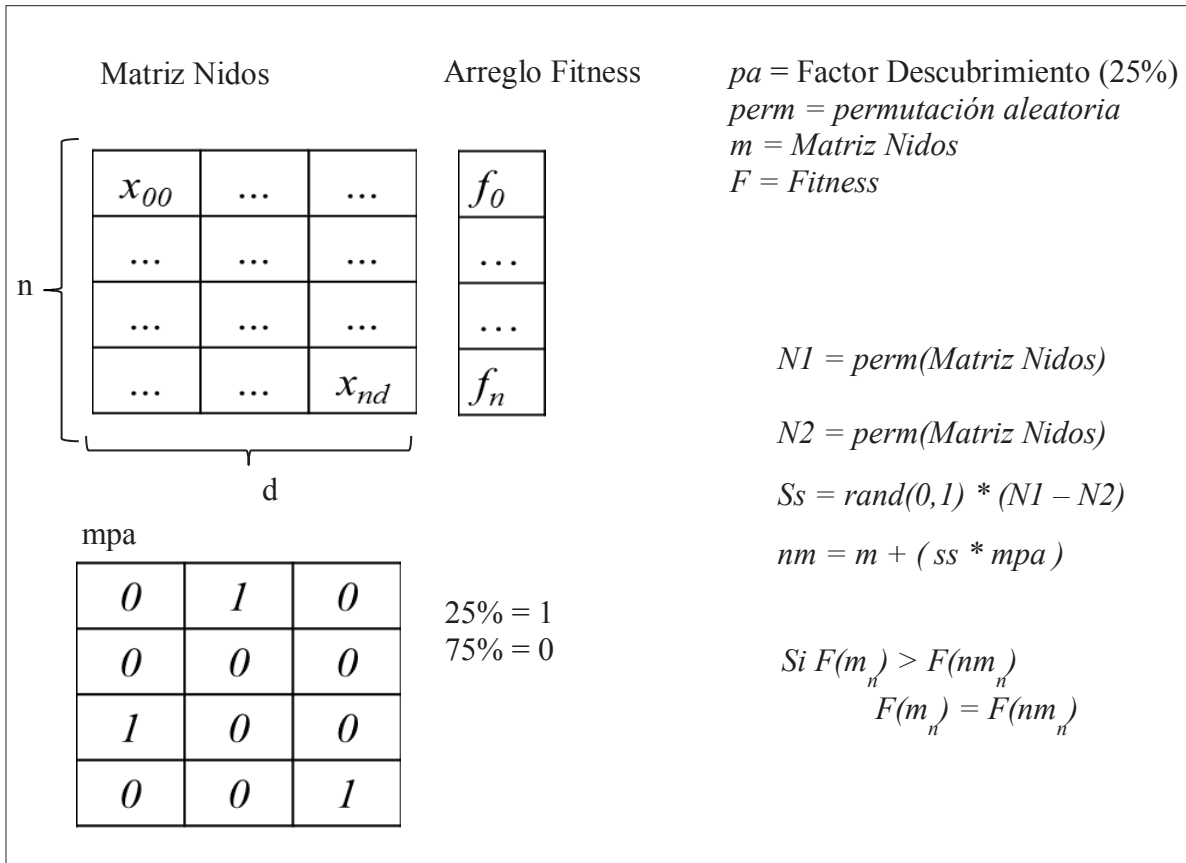


Figura 8.3 Búsqueda de nuevas soluciones

La Figura 8.3 muestra el proceso de búsqueda de nuevas soluciones utilizando permutaciones de la matriz original de soluciones, este proceso solo afecta el 25% de las soluciones existentes utilizando para ello la matriz mpa que es generada aleatoriamente y donde solo las casillas marcadas con el numero 1 son modificadas.

Los dos procesos finales descritos, son repetidos continuamente a fin de ir mejorando las soluciones con el paso de cada generación, una vez que se ha alcanzado el punto deseado de término, el algoritmo procede a entregar la mejor solución encontrada.

9 Pruebas

Una vez implementado, el algoritmo se somete a distintas pruebas para verificar su correcto funcionamiento, esto permite asegurar que las soluciones entregadas representan realmente una mejora en el proceso de búsqueda.

Debido a que la generación de nuevas soluciones depende en gran medida de la generación de números aleatorios, es necesario verificar que las funciones encargadas de generar estos valores cumplan con las características requeridas.

- La función “*rand.nextDouble()*” perteneciente a la clase “*java.util.Random*” es utilizada para generar valores aleatorios entre $[0,1]$, se espera que los valores generados por esta función se encuentren uniformemente distribuidos. Para evaluar esto se tomaron valores durante la ejecución y se generaron cinco muestras de diez mil valores, en base a estos se obtuvo una media de 0,500921401953774 ver Figura 8.1.

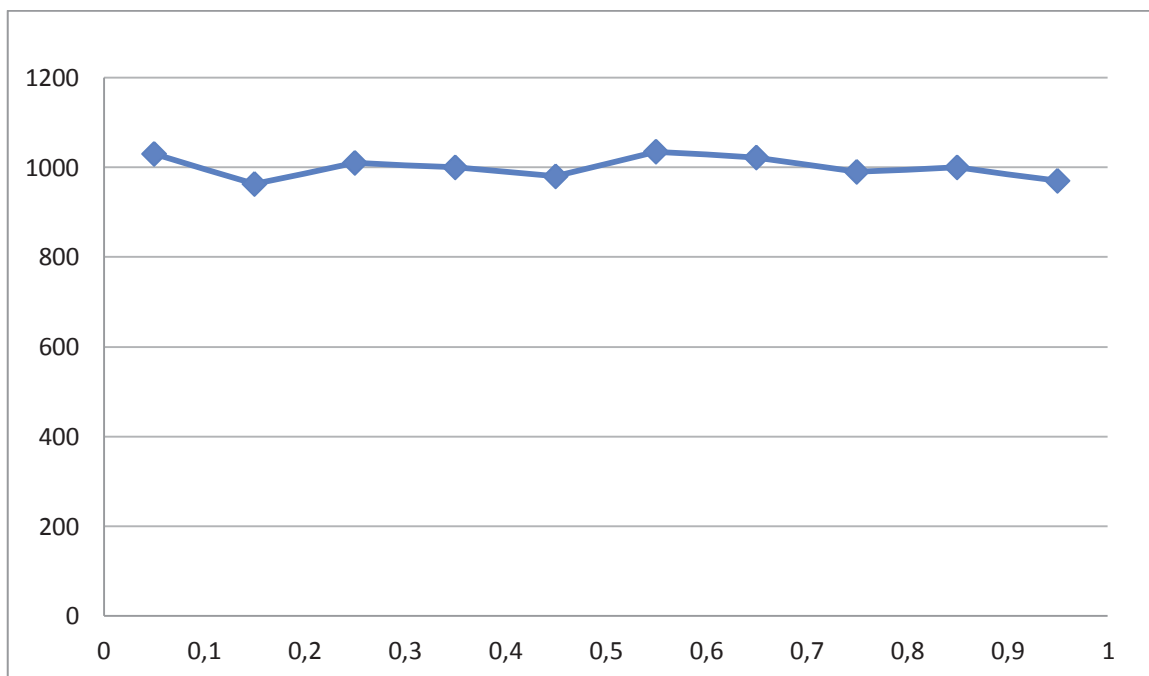


Figura 9.1 Función nextDouble()

El Figura 9.1 representa la distribución de una muestra aleatoria de 10.000 números, estos se distribuyeron en los intervalos graficados dejando en evidencia la uniformidad de la distribución, por lo que esta función sumada a la generación de semillas aleatorias, permite una fuente confiable de números aleatorios.

La función “*rand.nextGaussian()*” “*java.util.Random*” es utilizada para generar valores con una distribución normal, esto implica que para demostrar su calidad, se debe calcular la media y la desviación estándar de la muestra.

Tabla 9.1 Función nextGaussian

	Muestra de tamaño 10000 (Java)	Muestra de tamaño 10000 (Matlab)	Distribución normal
Media	0,001669874500633	0,013391484101464	0
Desviación Estándar	0,990690710457876	1,01169668122888	1

Luego de verificar los datos de la muestra, compararlos con los obtenidos por Matlab y los esperados Tabla 9.1, se puede comprobar que la muestra cumple con los requerimientos y por lo tanto se puede confiar en la función para la generación de valores aleatorios.

9.1 Pruebas de Ejecución

A continuación se presentan las pruebas preliminares de ejecución del algoritmo. Se busca identificar primero los parámetros y las funciones de selección que mejores resultados obtengan.

Las pruebas fueron realizadas sobre un equipo con las siguientes características, procesador Intel Core i3-2120 CPU 3.30 GHz, 4Gb Ram y Sistema Operativo Windows 7 32-bits.

Primero se realizaron pruebas sobre las Funciones de Selección (CF), la Tabla 9.2 muestra las 4 CF que se utilizaron para realizar las pruebas, estas pruebas tenían por objetivo definir cuál de las 4 CF presentaba los mejores resultados.

Tabla 9.2 Funciones de Selección.

	Parámetros		
CF1	B	STEP	PTAVFES
CF2	B	IN1	IN2
CF3	B	IN1	IN2 STEP
CF4	VF	DMAX	DB

La Tabla 9.3 muestra los parámetros que utilizan las CF, estos parámetros sin mediciones tomadas durante la resolución del problema y representan el estado en que se encuentra la búsqueda de una solución.

Tabla 9.3 Parámetros de la CF

Parámetro	Definición
B	“Backtracks”.
STEP	Cantidad de pasos.
PTAVFES	Porcentaje del total acumulado de variables instanciadas por enumeración en cada paso sobre el total de variables.
IN1	Variación de la profundidad previa y actual.
IN2	Relación de Profundidad actual o profundidad previa.
VF	Número de variables instanciadas.
DMAX	Profundidad máxima en el árbol de búsqueda.
DB	“Depth Back”.

Los resultados obtenidos se midieron en base a los dos indicadores de desempeño BT y “Solving Time” que representan el número total de “BackTracks” y la cantidad de tiempo utilizado en milisegundos (ms), utilizando estas dos medidas se define el desempeño de las Funciones de Selección, y en base a los resultados la Función de Selección 2 (CF2) representa la mejor alternativa y es la que será utilizada para la siguiente prueba.

Tabla 9.4 Comparación de CF

Problema	“Solving Time”(ms)				“BackTracks”			
	CF1	CF2	CF3	CF4	CF1	CF2	CF3	CF4
Q-8	283	266	354	281	4	4	5	4
Q-10	341	296	312	401	2	2	2	3
Q-12	467	374	418	384	3	1	3	2
Q-20	861	687	832	708	13	11	13	11
Q-50	3802	3635	3948	3690	195	177	209	183
Q-75	12485	11996	13646	13256	818	818	890	887
MS-4	624	468	501	575	1	0	0	1
MS-5	611	484	536	511	101	84	92	92
Sudoku	408	327	298	356	0	0	0	0
KT-5	2967	2730	2681	3008	767	767	767	767
KT-6	39609	36988	43178	38711	19458	18729	20416	18729
Promedio	5678	5296	6064	5626	1942	1872	2036	1880

Las pruebas sobre las Funciones de Selección, fueron realizadas utilizando los valores originales del algoritmo “Cuckoo Search”, los resultados obtenidos se muestran en la Tabla 9.4 donde queda claro que la Función de Selección 2 (CF2) es la que presenta un mejor desempeño.

Las pruebas para definir la mejor CF fueron realizadas utilizando los valores originales propuestos por el autor, estos valores incluyen el número de Nidos ($N = 15$), y el número de Iteraciones ($T = 10$).

Habiendo seleccionado la CF más adecuada para resolver problemas, es necesario calibrar los parámetros propios del algoritmo “*Cuckoo Search*” para poder asegurar resultados óptimos. Para encontrar estos valores, se realizó un nuevo conjunto de pruebas sobre los mismos problemas previamente descritos.

- N-Queens: $n = \{8, 10, 12, 20, 50, 75\}$.
- Magic Square: $n = \{4, 5\}$.
- Sudoku: $n = \{1\}$.
- Knight’s Tour: $n = \{5, 6\}$.

Cada una de las pruebas mencionadas se realizó utilizando todas las posibles combinaciones de los valores N y T que se presentan a continuación.

- Número de Nidos = $\{15, 25, 35, 50\}$.
- Número de Iteraciones = $\{10, 15, 20, 30, 40\}$.
- Factor de Descubrimiento = $\{0.25\}$.

Los resultados de estas pruebas dieron como conclusión que para la CF2, los parámetros que mejores resultados ofrecen son $N = 15$ y $T = 15$, el Factor de Descubrimiento no fue modificado ya que el autor identificó este valor como el óptimo para el algoritmo.

9.2 Pruebas Sobre las Estrategias de Enumeración

Como fue explicado en la sección 4, el objetivo de las Funciones de Selección es permitir elaborar un ranking de las mejores Estrategias de Enumeración, a fin de permitir que la herramienta obtenga los mejores resultados.

Un portafolio de 24 Estrategias de Enumeración (EE), permite a la herramienta tener una amplia selección de opciones a la hora de resolver un cierto problema, pero debido a las diferencias entre estas mismas, cada estrategia presenta un desempeño distinto al ser enfrentadas a un problema.

La Tabla 9.5 enumera las Heurísticas de Selección de Variables, mientras que la Tabla 9.6 muestra las Heurísticas de Selección de Valor, las posibles combinaciones entre estas dos heurísticas dan origen a las 24 Estrategias de Enumeración que forman parte del portafolio utilizado para resolver los problemas.

Tabla 9.5 Heurísticas de Selección de Variable.

Nombre	Descripción
First	Escoge la primera variable de la lista.
AMRV	Escoge la variable con el dominio más grande.
MRV	Escoge la variable con el dominio más pequeño.
Occurrence	Escoge la variable con mayor cantidad de restricciones.
Smallest	Escoge la variable con el valor más pequeño de dominio.
Largest	Escoge la variable con mayor valor en el dominio.
Most Constrained	Escoge la variable con el tamaño de dominio más pequeño.
Max Regret	Escoge la variable con la mayor diferencia entre el valor más pequeño y el más pequeño del segundo dominio.

Tabla 9.6 Heurísticas de Selección de Valor.

Nombre	Descripción
Indomain Min	Escoge el elemento más pequeño del dominio.
Indomain Middle	Escoge el elemento medio del dominio.
Indomain Max	Escoge el elemento más grande del dominio.

A fin de obtener un punto de partida, se realizaron pruebas sobre cada Estrategia de Enumeración, obteniendo resultados que permiten identificar que estrategias consiguen mejores resultados para un cierto problema en particular.

A continuación se presenta la tabla con los resultados obtenidos en las pruebas específicas para cada Estrategia de Enumeración, se consideraron principalmente dos indicadores para definir su desempeño, el total de “*BackTracks*” (TB) y el “*SolvingTime*” alcanzado durante la resolución de los problemas.

Cabe destacar que las pruebas se realizaron usando únicamente la Estrategia de Enumeración (EE) indicada, y que se consideró un tiempo máximo de 600 segundos antes de evaluar la prueba como “*Time Out*” (t.o.).

Tabla 9.7 “Solving Time” Queen N.

	Queen N – “SolvingTime” (ms)						
	Q-8	Q-10	Q-12	Q-15	Q-20	Q-50	Q-75
EE1	5	5	12	57	20405	t.o.	t.o.
EE2	5	8	11	903	4867	t.o.	t.o.
EE3	5	3	11	3	16	532	4280
EE4	4	4	11	59	20529	t.o.	t.o.
EE5	2	4	13	28	1294	t.o.	t.o.
EE6	4	5	14	79	26972	t.o.	t.o.
EE7	4	3	11	3	15	524	4217
EE8	2	4	10	24	93	t.o.	t.o.
EE9	5	5	11	58	20349	t.o.	t.o.
EE10	5	8	11	971	4780	t.o.	t.o.
EE11	4	7	11	3	18	532	4336
EE12	5	5	11	61	23860	t.o.	t.o.
EE13	2	5	13	29	1250	t.o.	t.o.
EE14	4	5	14	83	36034	t.o.	t.o.
EE15	4	3	11	3	17	533	4195
EE16	2	4	10	25	87	t.o.	t.o.
EE17	5	4	11	58	22286	t.o.	t.o.
EE18	5	7	10	953	4547	t.o.	t.o.
EE19	4	2	11	3	16	520	4334
EE20	4	4	11	59	13135	t.o.	t.o.
EE21	4	5	14	79	26515	t.o.	t.o.
EE22	2	4	13	28	1249	t.o.	t.o.
EE23	4	3	11	3	16	521	4187
EE24	2	5	8	102	1528	t.o.	t.o.

Tabla 9.8 “BackTracks” (TB) Queen N.

	Queen N – “Backtracks” (TB)						
	Q-8	Q-10	Q-12	Q-15	Q-20	Q-50	Q-75
EE1	10	6	15	73	10026	>121277	>118127
EE2	11	12	11	808	2539	>160845	>152812
EE3	10	4	16	1	11	177	818
EE4	10	6	15	73	10026	>121277	>118127
EE5	3	6	17	40	862	>173869	>186617
EE6	9	6	16	82	15808	>143472	>137450
EE7	10	4	16	1	11	177	818
EE8	3	5	12	28	63	>117616	>133184
EE9	10	6	15	73	10026	>121277	>118127
EE10	11	12	11	808	2539	>160845	>152812
EE11	10	12	16	1	11	177	818
EE12	10	6	15	73	10026	>121277	>118127
EE13	3	6	17	40	862	>173869	>186617
EE14	9	6	16	82	15808	>143472	>137450
EE15	10	4	16	1	11	177	818
EE16	3	5	12	28	63	>117616	>133184
EE17	10	6	15	73	10026	>121277	>118127
EE18	11	12	11	808	2539	>160845	>152812
EE19	10	4	16	1	11	177	818
EE20	10	6	15	73	10026	>121277	>118127
EE21	9	6	16	82	15808	>173869	>186617
EE22	3	6	17	40	862	>143472	>137450
EE23	10	4	16	1	11	177	818
EE24	2	37	13	127	1129	>117616	>133184

Tabla 9.9 “Solving Time” Magic Square.

	Magic Square – “<i>SolvingTime</i>” (ms)					
	MS-3	MS-4	MS-5	MS-6	MS-7	MS-8
EE1	1	14	1544	t.o.	t.o.	t.o.
EE2	5	2340	t.o.	t.o.	t.o.	t.o.
EE3	1	6	296	t.o.	t.o.	t.o.
EE4	1	21	6490	t.o.	t.o.	t.o.
EE5	1	21	t.o.	t.o.	t.o.	t.o.
EE6	4	1500	t.o.	t.o.	t.o.	t.o.
EE7	1	6	203	t.o.	t.o.	t.o.
EE8	1	11	1669	t.o.	t.o.	t.o.
EE9	1	13	1498	t.o.	t.o.	t.o.
EE10	4	2366	t.o.	t.o.	t.o.	t.o.
EE11	1	6	297	t.o.	t.o.	t.o.
EE12	1	21	6053	t.o.	t.o.	t.o.
EE13	1	21	t.o.	t.o.	t.o.	t.o.
EE14	4	1495	t.o.	t.o.	t.o.	t.o.
EE15	1	6	216	t.o.	t.o.	t.o.
EE16	1	11	1690	t.o.	t.o.	t.o.
EE17	1	88	t.o.	t.o.	t.o.	t.o.
EE18	1	37	t.o.	t.o.	t.o.	t.o.
EE19	1	99	t.o.	t.o.	t.o.	t.o.
EE20	1	42	165878	t.o.	t.o.	t.o.
EE21	1	147	t.o.	t.o.	t.o.	t.o.
EE22	1	37	153679	t.o.	t.o.	t.o.
EE23	1	102	t.o.	t.o.	t.o.	t.o.
EE24	1	79	t.o.	t.o.	t.o.	t.o.

Tabla 9.10 “BackTracks” (TB) Magic Square.

	Magic Square – “Backtracks” (TB)					
	MS-3	MS-4	MS-5	MS-6	MS-7	MS-8
EE1	0	12	910	>177021	>170762	>220259
EE2	4	1191	>191240	>247013	>39181	>238145
EE3	0	3	185	>173930	>169053	>195870
EE4	0	10	5231	>187630	>127273	>172751
EE5	0	22	>153410	>178895	>79041	>197492
EE6	4	992	>204361	>250986	>230408	>238472
EE7	0	3	193	>202927	>166861	>196884
EE8	0	13	854	>190877	>202632	>237239
EE9	0	12	910	>177174	>170762	>220259
EE10	4	1191	>191240	>247013	>63806	>238145
EE11	0	3	185	>174068	>169053	>195870
EE12	0	10	5231	>187777	>153768	>172751
EE13	0	22	>153410	>179026	>77935	>197492
EE14	4	992	>204361	>251193	>230408	>238472
EE15	0	3	193	>203089	>96730	>196884
EE16	0	13	854	>191042	>247427	>237239
EE17	1	51	>204089	>237428	>229035	>204107
EE18	0	42	>176414	>176535	>116846	>83366
EE19	1	3	>197512	>231600	>185681	>146714
EE20	1	29	74063	>190822	>187067	>172010
EE21	1	95	>201698	>239305	>249686	>247577
EE22	0	46	74711	>204425	>46233	>127685
EE23	1	96	>190692	>204119	>130196	>193113
EE24	1	47	>183580	>214287	>219116	>177117

Tabla 9.11 “Solving Time” y “BackTracks” (TB) Latin Square.

	Latin Square – “SolvingTime” (ms)					Latin Square – “Backtracks” (TB)				
	LS-4	LS-5	LS-6	LS-7	LS-8	LS-4	LS-5	LS-6	LS-7	LS-8
EE1	2	3	5	9	11	0	0	0	12	0
EE2	2	11	102	t.o.	14	0	9	163	>99332	0
EE3	1	3	5	7	12	0	0	0	0	0
EE4	1	3	5	9	12	0	0	0	14	0
EE5	2	8	60	t.o.	14	0	7	61	>99403	0
EE6	2	4	7	88	12	0	0	0	71	0
EE7	1	2	4	7	12	0	0	0	0	0
EE8	1	2	5	7	12	0	0	0	0	0
EE9	2	3	5	13	11	0	0	0	9	0
EE10	1	11	103	t.o.	14	0	9	163	>99332	0
EE11	2	3	6	8	12	0	0	0	0	0
EE12	2	3	6	14	13	0	0	0	9	0
EE13	2	8	62	t.o.	15	0	7	61	>99539	0
EE14	2	4	7	92	14	0	0	0	71	0
EE15	2	3	5	9	14	0	0	0	0	0
EE16	2	3	5	9	14	0	0	0	0	0
EE17	2	3	6	14	12	0	0	0	9	0
EE18	2	12	107	t.o.	14	0	9	163	>99481	0
EE19	2	3	5	8	12	0	0	0	0	0
EE20	2	4	5	16	13	0	0	0	9	0
EE21	2	4	7	93	13	0	0	0	71	0
EE22	2	8	64	t.o.	15	0	7	61	>99539	0
EE23	2	3	6	8	13	0	0	0	0	0
EE24	2	3	5	10	17	0	0	0	0	1

Tabla 9.12 “Solving Time” y “BackTracks” (TB) Sudoku.

	Sudoku – “ <i>SolvingTime</i> ” (ms)					Sudoku – “ <i>Backtracks</i> ” (TB)				
	S-1	S-2	S-5	S-7	S-9	S-1	S-2	S-5	S-7	S-9
EE1	5	35	7453	26882	4	0	18	4229	10786	0
EE2	2247	30515	t.o.	t.o.	209	1523	10439	>89125	>59828	114
EE3	6	10	2181	2135	8	0	4	871	773	0
EE4	6	50	8274	25486	5	0	18	4229	10786	0
EE5	18	225	t.o.	t.o.	5	7	155	>112170	>81994	0
EE6	6	1607	t.o.	t.o.	3	0	764	>83735	>80786	0
EE7	6	10	2247	2187	8	0	4	871	773	0
EE8	8	10	897	31732	8	0	2	308	10379	0
EE9	4	35	7521	26621	4	0	18	4229	10786	0
EE10	4754	29797	t.o.	t.o.	215	1523	10439	>89125	>59828	114
EE11	6	10	2394	2069	8	0	4	871	773	0
EE12	6	50	9015	26573	5	0	18	4229	10786	0
EE13	18	225	t.o.	t.o.	4	7	155	>112174	>81994	0
EE14	7	1732	t.o.	t.o.	3	0	764	>83735	>80786	0
EE15	6	10	2310	2094	8	0	4	871	773	0
EE16	8	10	972	30767	8	0	2	308	10379	0
EE17	8	5	t.o.	3725	6	3	2	>104148	1865	1
EE18	2497	18836	t.o.	t.o.	20	8482	6541	>80203	>80295	7
EE19	15	30	2590	338	7	0	9	963	187	0
EE20	15	5	t.o.	5350	8	3	2	>104148	1865	1
EE21	74	100	t.o.	t.o.	4	49	89	>78774	>93675	0
EE22	3615	1710	t.o.	t.o.	4	1039	887	>101058	>91514	0
EE23	15	30	2670	378	8	4	9	963	187	0
EE24	125	40	t.o.	9168	6	72	12	>92557	2626	0

Tabla 9.13 “Solving Time” y “BackTracks” (TB) Quasi Group.

	Quasi Group – “ <i>SolvingTime</i> ” (ms)					Quasi Group – “ <i>Backtracks</i> ” (TB)				
	QG-1	QG-3	QG-5	QG-6	QG-7	QG-1	QG-3	QG-5	QG-6	QG-7
EE1	1	1	t.o.	45	256	0	0	>145662	30	349
EE2	1	1	t.o.	t.o.	8020	0	0	>103603	>176613	3475
EE3	1	1	7510	15	10	0	0	8343	0	1
EE4	1	1	t.o.	45	307	0	0	>145656	30	349
EE5	1	1	t.o.	t.o.	943	1	0	>92253	>83087	4417
EE6	1	1	t.o.	3605	16896	0	0	>114550	965	4417
EE7	1	1	9465	15	10	0	0	8343	0	1
EE8	2	1	t.o.	t.o.	16	1	0	>93315	>96367	4
EE9	2	1	t.o.	40	240	1	0	>145835	30	349
EE10	1	1	t.o.	t.o.	13481	0	0	>103663	>176613	3475
EE11	1	1	9219	15	10	0	0	8343	0	1
EE12	1	1	t.o.	45	348	0	0	>145830	30	349
EE13	1	1	t.o.	t.o.	1097	1	0	>92355	>83087	583
EE14	1	1	t.o.	3565	18205	0	0	>114550	965	4417
EE15	1	1	10010	15	11	0	0	8343	0	1
EE16	2	1	t.o.	t.o.	15	1	0	>93315	>93820	4
EE17	1	1	9743	7075	9	0	0	7743	2009	3
EE18	1	1	t.o.	t.o.	1878	0	0	>130635	>75475	845
EE19	1	1	20	125	12	0	0	0	89	1
EE20	1	1	10507	6945	9	0	0	7763	2009	3
EE21	1	1	t.o.	t.o.	1705	0	0	>96083	>108987	773
EE22	1	1	t.o.	t.o.	9	0	0	>94426	>124523	1
EE23	1	1	21	130	12	0	0	0	89	1
EE24	1	1	t.o.	t.o.	14	0	0	>95406	>89888	1

Tabla 9.14 “Solving Time” y “BackTracks” (TB) Knight Tour.

	Knight Tour – “SolvingTime” (ms)		Knight Tour – “Backtracks” (TB)	
	KT-5	KT-6	KT-5	KT-6
EE1	1825	90755	767	37695
EE2	t.o	t.o	>179097	>177103
EE3	2499	111200	767	37695
EE4	t.o	89854	>97176	35059
EE5	t.o	t.o	>228316	>239427
EE6	t.o	t.o	>178970	>176668
EE7	t.o	39728	>73253	14988
EE8	t.o	t.o	>190116	>194116
EE9	1908	93762	767	37695
EE10	t.o	t.o	>179126	>177129
EE11	2625	102387	767	37695
EE12	t.o	109157	>97176	35059
EE13	t.o	t.o	>228316	>239427
EE14	t.o	t.o	>178970	>176668
EE15	t.o	46673	>73253	14998
EE16	t.o	t.o	>190116	>194116
EE17	1827	96666	767	37695
EE18	t.o	t.o	>179126	>177129
EE19	2620	97388	767	37695
EE20	t.o	90938	>97178	35059
EE21	t.o	t.o	>178970	>176668
EE22	t.o	t.o	>228316	>239427
EE23	t.o	40997	>73253	14998
EE24	t.o	t.o	>190116	>160789

Tabla 9.15 “Solving Time” y “BackTracks” (TB) Langford.

	Lagnford (2-N) – “SolvingTime” (ms)					Lagnford (2-N) – “Backtracks” (TB)				
	N=8	N=12	N=16	N=20	N=23	N=8	N=12	N=16	N=20	N=23
EE1	3	20	70	191	79	2	16	39	77	26
EE2	10	242	70526	t.o.	t.o.	15	223	24310	>158157	>157621
EE3	3	4	231	546	286	2	1	97	172	64
EE4	4	29	115	318	140	2	16	39	77	26
EE5	3	43	1217	61944	68254	2	29	599	26314	29805
EE6	3	32	489	11	19	1	22	210	1	3
EE7	3	4	237	553	285	2	1	97	172	64
EE8	2	22	7	240	19	0	12	0	64	7
EE9	3	20	69	185	79	2	16	39	77	26
EE10	10	270	55291	t.o.	t.o.	15	223	24310	>158157	>157621
EE11	4	4	250	538	285	2	1	97	172	64
EE12	4	29	118	312	140	2	16	39	77	26
EE13	3	44	1273	61345	71209	2	29	599	26314	29805
EE14	3	32	530	11	19	1	22	210	1	3
EE15	3	5	235	541	278	2	1	97	172	64
EE16	2	21	8	237	19	0	12	0	64	7
EE17	3	18	66	170	75	2	16	39	77	26
EE18	11	242	55687	t.o.	t.o.	15	223	24592	>158028	>157649
EE19	3	4	245	562	272	2	1	98	172	64
EE20	4	29	107	294	126	2	16	39	77	26
EE21	3	33	510	11	20	1	22	210	1	3
EE22	3	43	1297	58732	73168	2	29	599	26314	29805
EE23	3	5	240	569	276	2	1	98	172	64
EE24	5	13	584	15437	10	4	6	239	4521	0

9.3 Pruebas Sobre Cuckoo Search y Comparaciones

Teniendo en consideración las pruebas expuestas en la sección 8.1 que permitieron definir los valores propios del algoritmo Cuckoo Search para obtener los mejores resultados, y teniendo como base las pruebas realizadas sobre las Estrategias de Enumeración expuestas en la sección 9.2, se procede a realizar las pruebas sobre el algoritmo propuesto a fin de analizar su desempeño.

Para las pruebas se consideraran el uso de la Función de Selección (CF2), los valores $N= 15$, $T= 15$ y $P_a= 0.25$. Con respecto a las Estrategias de Enumeración, se utilizarán las 24 estrategias presentes, y solo se desactivaran selectivamente cuando el algoritmo sea incapaz de resolver el problema, momento en el cual se desactivaran aquellas estrategias que obtuvieron resultados de “*Time Out*” durante las pruebas individuales.

Junto con los resultados obtenidos por el algoritmo propuesto “*Cuckoo Search*” (CS), se incluyen los resultados obtenidos por otros dos algoritmos, “*Artificial Bee Colony*” (ABC) y “*Bat Inspired Algorithm*” (BAT), “*Gravitational Search Algorithm*” (GSA), “*Particle Swarm Optimization*” (PSO) y Algoritmo Genético a modo de comparación.

Las pruebas fueron realizadas sobre equipos de iguales características para permitir una correcta comparación entre los tres algoritmos de optimización, Intel Core i3-2120 CPU 3.30 GHz, 4Gb Ram y Sistema Operativo Windows 7 32-bits.

A continuación se presenta la tabla comparativa con los resultados de las pruebas realizadas.

Tabla 9.16 Resultados y comparación de “Solving Time” entre algoritmos.

	“Solving Time” (ms)					
	CS	ABC	BAT	GSA	PSO	Genético
Q-8	290	230	501	625	4982	645
Q-10	310	265	604	779	7735	735
Q-12	380	315	750	972	24369	875
Q-15	470	380	1024	1322	10483	972
Q-20	685	575	1556	2046	52827	7520
Q-50	3269	2770	7856	10760	1480195	6530
Q-75	12981	10775	20870	28254	1,20E+14	16069
MS-3	340	290	545	731	2745	735
MS-4	485	400	902	1053	15986	1162
MS-5	590	340	775	1004	565155	1087
MS-6	1135	868	1038	1526	t.o.	t.o.
MS-7	t.o.	2613	1189	t.o.	t.o.	t.o.
MS-8	t.o.	t.o.	t.o.	t.o.	t.o.	t.o.
S-1	355	300	651	879	12043	2270
S-2	330	287	586	760	10967	15638
S-5	245	335	628	836	2679975	8202
S-7	555	465	593	847	967014	25748
S-9	330	265	608	841	10351	740
K-5	1429	1132	3194	4383	4563751	21089
K-6	7575	1961	4570	6508	2,03E+14	170325
LS-4	210	180	333	425	3323	695
LS-5	230	195	387	482	6647	692
LS-6	250	215	438	556	12716	725
LS-7	275	245	496	610	20519	777
LS-8	310	265	560	733	31500	752
QG-1	195	171	262	295	2160	675
QG-3	160	158	214	254	1250	665
QG-5	295	312	645	824	59158	11862
QG-6	345	310	626	845	44565	947
QG-7	315	262	516	777	28612	795
LF 2-8	255	219	415	556	5000	762
LF 2-12	310	265	506	679	10430	1212
LF 2-16	440	279	623	829	20548	1502
LF 2-20	560	337	742	942	28466	1409
LF 2-23	505	360	810	1054	30468	1287
x	1103,4	833,5	1647,5	2242,1	10084267834811	9534,4

Tabla 9.17 Resultados y comparación de “Back Tracks” entre algoritmos.

	“BackTracks”					
	CS	ABC	BAT	GSA	PSO	Genético
Q-8	3	1	2	1	3	1
Q-10	3	1	4	0	1	4
Q-12	3	3	0	0	1	40
Q-15	1	1	1	1	1	68
Q-20	10	11	8	2	11	3879
Q-50	1	0	0	0	0	15
Q-75	818	797	31	2	818	17
MS-3	0	0	0	0	0	0
MS-4	1	2	3	0	0	42
MS-5	26	16	99	12	14	198
MS-6	257	303	733	734	>47209	>176518
MS-7	1339	876	1445	1874	>56342	>213299
MS-8	>191769	>107697	>90142	>323094	>61786	>225158
S-1	0	0	0	0	0	732
S-2	4	1	1	2	2	6541
S-5	308	17	180	37	13	4229
S-7	93	105	117	292	256	10786
S-9	0	0	0	0	0	7
K-5	4	1	7	1	106	50571
K-6	2196	90	1499	3284	12952	0
LS-4	0	0	0	0	0	0
LS-5	0	0	0	0	0	0
LS-6	0	0	0	0		0
LS-7	0	0	0	0	0	0
LS-8	0	0	0	0	0	0
QG-1	0	0	0	0	0	0
QG-3	0	0	0	0	0	0
QG-5	0	1	0	0	0	7763
QG-6	1	0	0	0	0	0
QG-7	0	1	1	0	0	4
LF 2-8	2	0	0	0	0	15
LF 2-12	0	1	1	1	1	223
LF 2-16	50	0	0	0	0	97
LF 2-20	47	1	1	1	1	64
LF 2-23	19	0	0	0	3	0
x	152,6	65,6	121,6	183,7	457,6	2665,5

10 Conclusiones

La programación con restricciones ofrece una alternativa al enfoque clásico de programación lógica, ofreciendo un modelo de variables y restricciones que permite modelar de forma natural ciertos problemas.

Si bien existen métodos para optimizar el proceso de resolución de problemas, la necesidad de contar con un usuario experto limita en gran medida la posibilidad de aprovechar al máximo los beneficios de la programación con restricciones.

La integración de la Programación con restricciones y la Búsqueda Autónoma brinda la posibilidad de obtener un sistema capaz de adecuarse a la situación de forma automática, entregando un mejor desempeño.

Cuckoo Search es un algoritmo meta heurístico sencillo pero robusto, un candidato prometedor para ser implementado como optimizador en la integración entre CP y AS que potencialmente entregara resultados favorables a la hora de optimizar el proceso de búsqueda.

Uno de los mayores beneficios en la implementación del algoritmo, es la necesidad de solo definir dos variables, el tamaño de la población y el porcentaje de eliminación, esto facilita el proceso de ajustar estos valores y hace de este algoritmo uno muy robusto y de sencilla adaptación a múltiples problemas.

Luego de realizar pruebas para definir las CF y los parámetros específicos para el algoritmo “*Cuckoo Search*”, se evaluaron los resultados obtenidos en las pruebas formales y se compararon con otros algoritmos.

Las Tablas 9.16 y 9.17 muestran como *Cuckoo Search* obtuvo el segundo mejor lugar en base al rendimiento, siendo solo superado por *Artificial Bee Colony*, pero demostrando una clara mejora en relación a los otros algoritmos.

Se espera que el lector haya comprendido como búsqueda autónoma puede mejorar la capacidad de resolver problemas de la programación con restricciones, como el algoritmo propuesto puede presentar resultados prometedores al ser implementado como optimizador, la forma en que el algoritmo propuesto es incorporado a arquitectura presentada y los resultados obtenidos al realizar pruebas sobre distintos problemas.

11 Referencias

- [1] B. Crawford, R. Soto, C. Castro, and E. Monfroy, *A Hyperheuristic Approach for Dynamic Enumeration Strategy Selection in Constraint Satisfaction*, International Work-conference on the Interplay Between Natural and Artificial Computation (IWINAC), pp. 295-304, 2011.
- [2] R. Krzysztof, *Principles of Constraint Programming*, Cambridge University Press, pp. 1-407, 2003.
- [3] Y. Hamadi, F. Saubion, and E. Monfroy, *Autonomous Search*, 1st ed.: Springer-Verlag Berlin Heidelberg, 2012.
- [4] X. Yang and S. Deb, *Cuckoo Search via Levy Flights*, World Congress on Nature & Biologically Inspired Computing (NaBIC), pp. 210-214, 2009.
- [5] S. Epstein, E. Freuder, R. Wallace, A. Morozov, and B. Samuels, *The Adaptive Constraint Engine*, In: Van Hentenryck, P. (ed.) CP 2002. LNCS, vol. 2470, pp. 525-542. Springer, Heidelberg (2002).
- [6] R. Soto, B. Crawford, E. Monfroy, and V. Bustos, *Using Autonomous Search for Generating Good Enumeration Strategy Blends in Constraint Programming*, International Conference of Computational Science and Its Applications (ICCSA), pp. 607-617, 2012.
- [7] B. Crawford, C. Castro, and E. Monfroy, *Using a choice function for guiding enumeration in constraint solving*, In: Proceedings of the 9th Mexican International Conference on Artificial Intelligence (MICAI), pp. 37-42. IEEE Computer Society (2010).
- [8] B. Crawford, R. Soto, M. Montecinos, C. Castro, and E. Monfroy, *A Framework for Autonomous Search in the Eclipse Solver*, Proceedings of the 24th International Conference on Industrial, Engineering and Other Applications of Applied Intelligent Systems IEA/AIE, Springer., no. 6703, pp. 79-84, 2011.
- [9] B. Crawford, R. Soto, C. Castro, and E. Monfroy, *Extensible CP-Based Autonomous Search*, In: Stephanidis, C. (ed.) Posters, HCII 2011, Part I. CCIS, vol. 173, pp. 561-565. Springer, Heidelberg (2011).

- [10] R. Soto, B. Crawford, C. Galleguillos, E. Monfroy, and F. Paredes, *A Pre-filtered Cuckoo Search Algorithm with Geometric Operators for Solving Sudoku Problems*, The Scientific World Journal, vol. 2014, no. 465359, p. 12, 2014.
- [11] B. Crawford et al., *Parameter tuning of a choice-function based hyperheuristic using Particle Swarm Optimization*, Expert Systems with Applications, vol. 40(5), pp. 1690-1695, 2013.
- [12] B. Crawford, R. Soto, R. Castro, E. Monfroy, and F. Paredes, *An Extensible Autonomous Search Framework for Constraint Programming*, Int. J. Phys. Sci., no. 614, pp. 3369-3376, 2011.
- [13] G. Viswanathan, V. Afanasyev, and S. Buldyrev, *Levy flight search patterns of wandering albatrosses*, Letters to nature, no. 381, pp. 413 - 415, Mayo 1996.
- [14] P. Civicioglu and E. Besdok, *A conceptual comparison of the Cuckoo-search, particle swarm optimization, differential evolution and artificial Bee Colony*, Artificial Intelligence Review, vol. 39, no. 4, pp. 315-346, Abril 2013.
- [15] X. Yang, *Nature Inspired Metaheuristic Algorithms*, segunda ed., ISBN-13: 978-1-905986-28-6, Ed. United Kingdom, University of Cambridge, United Kingdom: Luniver Press, 2010.